

Semirings for database provenance

Yiannis Hadjicharalambous

16 January 2026

What is database provenance?

Provenance describes the origins and the history of data in its life cycle. Such information is important to many data management tasks.

What is database provenance?

Provenance describes the origins and the history of data in its life cycle. Such information is important to many data management tasks.

Provenance is typically represented as a general form of annotation information, usually by tagging data on the tuple-level or value-level of a relation.

Types of provenance

- Which-provenance

Types of provenance

- Which-provenance
- Why-provenance

Types of provenance

- Which-provenance
- Why-provenance
- How-provenance

Types of provenance

- Which-provenance
- Why-provenance
- How-provenance

We need a unifying formal framework.

Example

Edges

from	to	
<i>a</i>	<i>a</i>	p
<i>a</i>	<i>b</i>	q
<i>b</i>	<i>a</i>	r
<i>b</i>	<i>c</i>	s

Example

Query: Where can I get to in 3 hops?

Edges

from	to	
<i>a</i>	<i>a</i>	p
<i>a</i>	<i>b</i>	q
<i>b</i>	<i>a</i>	r
<i>b</i>	<i>c</i>	s

Example

Edges

from	to	
<i>a</i>	<i>a</i>	p
<i>a</i>	<i>b</i>	q
<i>b</i>	<i>a</i>	r
<i>b</i>	<i>c</i>	s

Query: Where can I get to in 3 hops?

Provenance: *Which* edges do I use?

Example

Edges

from	to	
<i>a</i>	<i>a</i>	p
<i>a</i>	<i>b</i>	q
<i>b</i>	<i>a</i>	r
<i>b</i>	<i>c</i>	s

Query: Where can I get to in 3 hops?

Provenance: *Which* edges do I use?

I can get to (a, a) by using p three times.

Example

Edges

from	to	
<i>a</i>	<i>a</i>	p
<i>a</i>	<i>b</i>	q
<i>b</i>	<i>a</i>	r
<i>b</i>	<i>c</i>	s

Query: Where can I get to in 3 hops?

Provenance: *Which* edges do I use?

I can get to (a, a) by using p three times.

I can also get to (a, a) by using p then q and then r .

Example

Edges

from	to	
<i>a</i>	<i>a</i>	p
<i>a</i>	<i>b</i>	q
<i>b</i>	<i>a</i>	r
<i>b</i>	<i>c</i>	s

Query: Where can I get to in 3 hops?

Provenance: *Which* edges do I use?

I can get to (a, a) by using p three times.

I can also get to (a, a) by using p then q and then r .

Finally, I can get to (a, a) by using q then r and then p .

Semiring framework

Main idea: have the values for the annotations come from a semiring.

Semiring framework

Main idea: have the values for the annotations come from a semiring.
The most general of all being the polynomial semiring $(\mathbb{N}[X], +, \cdot, 0, 1)$.

Semiring framework

Main idea: have the values for the annotations come from a semiring.
The most general of all being the polynomial semiring $(\mathbb{N}[X], +, \cdot, 0, 1)$.
The provenance for (a, a) from our query from the previous slide would be the polynomial $p^3 + 2pqr$.

Positive relational algebra

The original paper translated the following relational algebra operations into the semiring framework with the $+$ and $\cdot$ operations:

The original paper translated the following relational algebra operations into the semiring framework with the $+$ and $\cdot$ operations:

- Selection (filter tuples): $\sigma_{\theta}(R)$

The original paper translated the following relational algebra operations into the semiring framework with the $+$ and $\cdot$ operations:

- Selection (filter tuples): $\sigma_{\theta}(R)$
- Projection (remove/rearrange attributes): $\pi_L(R)$

The original paper translated the following relational algebra operations into the semiring framework with the $+$ and $\cdot$ operations:

- Selection (filter tuples): $\sigma_{\theta}(R)$
- Projection (remove/rearrange attributes): $\pi_L(R)$
- Cartesian product: $R \times S$

The original paper translated the following relational algebra operations into the semiring framework with the $+$ and $\cdot$ operations:

- Selection (filter tuples): $\sigma_{\theta}(R)$
- Projection (remove/rearrange attributes): $\pi_L(R)$
- Cartesian product: $R \times S$
- Union (add tuples): $R \cup S$

The original paper translated the following relational algebra operations into the semiring framework with the $+$ and $\cdot$ operations:

- **Selection** (filter tuples): $\sigma_{\theta}(R)$
- **Projection** (remove/rearrange attributes): $\pi_L(R)$
- **Cartesian product**: $R \times S$
- **Union** (add tuples): $R \cup S$

Extending the semiring framework

Extensive work has been carried out to extend the semiring framework to more expressive query languages with:

Extending the semiring framework

Extensive work has been carried out to extend the semiring framework to more expressive query languages with:

- Recursion

Extending the semiring framework

Extensive work has been carried out to extend the semiring framework to more expressive query languages with:

- Recursion
- Aggregation

Extending the semiring framework

Extensive work has been carried out to extend the semiring framework to more expressive query languages with:

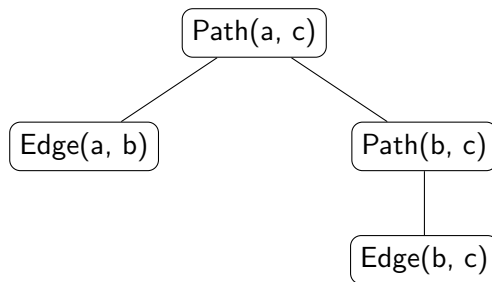
- Recursion
- Aggregation
- Negation

Datalog introduces recursion to our queries. This is the transitive closure of the relation Edge in Datalog:

Datalog introduces recursion to our queries. This is the transitive closure of the relation Edge in Datalog:

$$\text{Path}(x, y) \leftarrow \text{Edge}(x, y)$$
$$\text{Path}(x, z) \leftarrow \text{Edge}(x, y), \text{Path}(y, z)$$

Datalog introduces recursion to our queries. This is the transitive closure of the relation Edge in Datalog:

$$\text{Path}(x, y) \leftarrow \text{Edge}(x, y)$$
$$\text{Path}(x, z) \leftarrow \text{Edge}(x, y), \text{Path}(y, z)$$


Datalog provenance depends on a query's derivation trees. But we could have infinitely many derivations for a tuple and hence infinite provenance.

Datalog provenance depends on a query's derivation trees. But we could have infinitely many derivations for a tuple and hence infinite provenance.

We shall consider semirings of formal power series that are shown to be generated by finite algebraic systems of fixed point equations.

Aggregate queries

Books

title	genre	qty
book 1	adventure	4
book 2	fantasy	5
book 3	romance	6
book 4	adventure	3
book 5	fantasy	1
book 6	romance	2

GROUP BY genre

genre	total
adventure	7
fantasy	6
romance	8

SUM(qty)

Aggregation provenance

To accomplish provenance on aggregate queries, we move from only annotating tuples to also annotating the individual values within those tuples.

Aggregation provenance

To accomplish provenance on aggregate queries, we move from only annotating tuples to also annotating the individual values within those tuples.

To accommodate the aggregation operations and capture the interaction between tuple-level and aggregate-level annotations, we need to add more structure to our framework.

Negation in provenance

Negation is particularly tricky due to its non-monotonicity. With negation, provenance must also account for the absence of data, not just its presence. Such absence depends on the entire database state rather than on local witnesses.

Negation in provenance

Negation is particularly tricky due to its non-monotonicity. With negation, provenance must also account for the absence of data, not just its presence. Such absence depends on the entire database state rather than on local witnesses.

Adding negation conflicts with the compositional and local nature of semiring-based provenance. Arbitrary semirings also lack a general notion of subtraction or complement.

Approaches to negation

Several researchers proposed different ways to define negation on semiring databases.

Approaches to negation

Several researchers proposed different ways to define negation on semiring databases.

- Specifying a set of equations

Approaches to negation

Several researchers proposed different ways to define negation on semiring databases.

- Specifying a set of equations
- Monus operation on naturally ordered semirings

Approaches to negation

Several researchers proposed different ways to define negation on semiring databases.

- Specifying a set of equations
- Monus operation on naturally ordered semirings
- Taking equivalence classes

Open challenges

- Unified framework for recursion, aggregates, and negation

Open challenges

- Unified framework for recursion, aggregates, and negation
- Trade-offs between generality and usability

- Unified framework for recursion, aggregates, and negation
- Trade-offs between generality and usability
- Limits of semiring-based provenance